

33326460_NguyenKhang_Assignment4_TaskC&D

2026-06-03

GENERATIVE AI DECLARATION:

In this assignment, I used Gemini, ChatGPT Chatbot, improve writing and clarity, explain the coding, concepts, and suggest how to approach the implementation.

Import data set and load library

```
# Load tidyverse
library(tidyverse)
```

TASK C

```
# import the data set for task C
vic_prop <- read_csv('TaskC_property_victoria.csv')
```

Data wrangling

```
#Tidy session for the dataset
# Area is minus, some NA cell,
vic_prop_modified <- vic_prop %>%

# 1. Ignore rows where area is negative, zero, or too smalls (minimum 18 m^2)
# 2. Remove rows where area is NA
filter(
  !is.na(area),
  !is.na(ID),
  !is.na(postcode),
  !is.na(suburb),
  !is.na(sold_time),
  !is.na(sold_price),
  !is.na(address),
  !is.na(beds),
  !is.na(baths),
  !is.na(parking),
  !is.na(property_type),
  !is.na(description),
  area > 0, #minimum Land size should be more than 0
  sold_price > 0 # sold price should be high than 0
)
```

Explanation: This section focuses on data wrangling before proceeding to the analysis below. All columns with NA values are cleaned, which helps ensure accuracy for each question. The sold_price must be greater than zero for Questions 2, 3, 4, and 6 to be accurate. Finally, the area must also be greater than zero for Questions 3 and 5.

Question 1

```

#Q1: top 3 suburbs with the highest number of property transactions over the years, and plot their monthly transaction counts for the year 2022. Include Toorak in the plot as well, if it is not among the top 3 suburbs.
# Sort the 3 suburbs with the highest number of property and add Toorak if appropriate
suburbs_q1 <- vic_prop_modified %>%
  count(suburb, sort = TRUE) %>%
  slice_max(n, n = 3) %>%
  pull(suburb) %>%
  c("Toorak") %>%
  unique

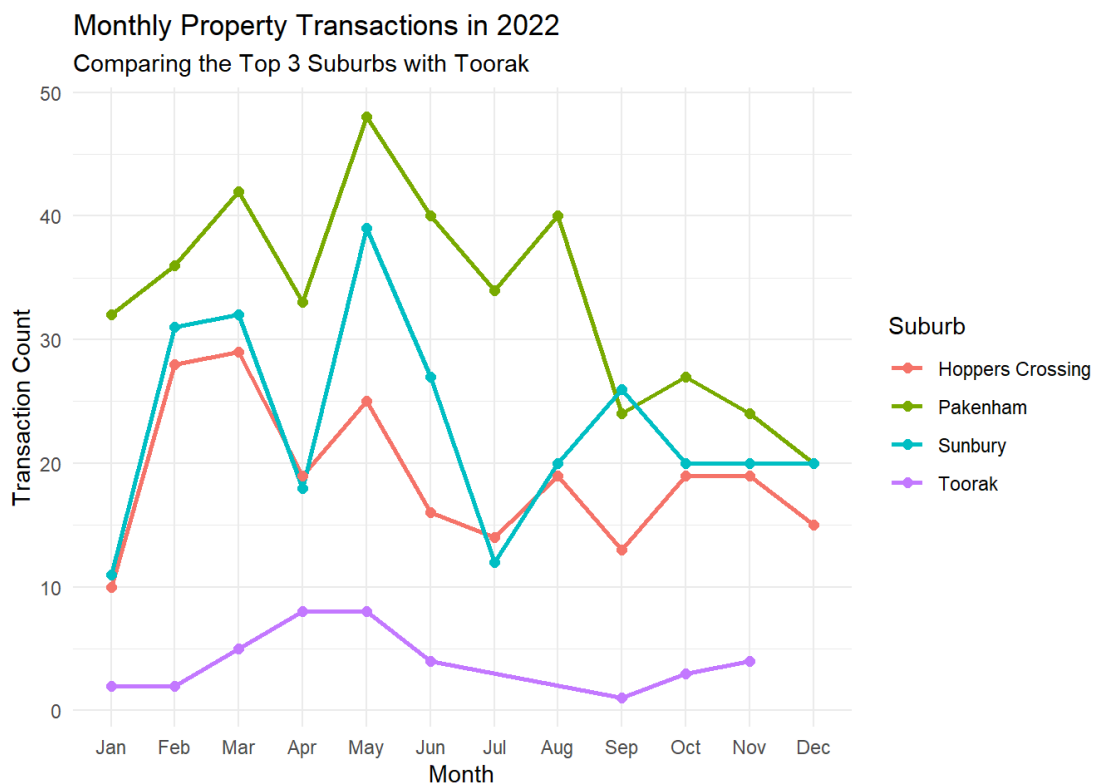
vic_prop_modified %>%
  # Identify top 3 suburbs overall + Toorak, and filter the dataset for them
  filter(suburb %in% suburbs_q1) %>%

  # Filter specifically for transactions in 2022
  filter(year(sold_time) == 2022) %>%
  mutate(month = month(sold_time, label = TRUE, abbr = TRUE)) %>%
  # Count the number of transactions per suburb, per month
  count(suburb, month) %>%
  ungroup() %>%
  # Plot session
  ggplot(aes(x = month, y = n, color = suburb, group = suburb)) +

  # geom_line() and geom_point() create the trend lines perfectly!
  geom_line(size = 1) +
  geom_point(size = 2) +

  # Descriptive Labels
  labs(
    title = "Monthly Property Transactions in 2022",
    subtitle = "Comparing the Top 3 Suburbs with Toorak",
    x = "Month",
    y = "Transaction Count",
    color = "Suburb"
  ) +
  theme_minimal()

```



Explanation: This code finds the top three suburbs with the most property sales and adds Toorak if it is not already on the list. It then filters the data for the year 2022, extracts the month from sold_time, aggregates the monthly transactions, and plots a line chart.

Question 2

#Q2: Top 3 keywords impact prices?

set.seed(123) #ensures the 10% random sample is reproducible.

```
vic_prop_modified %>%  
  # Extract a 10% random sample  
  slice_sample(prop = 0.1) %>%  
  
  # Select only the necessary columns to save memory  
  select(description, sold_price) %>%  
  
  # Convert text to lowercase and extract words (Letters only)  
  mutate(description = str_to_lower(description)) %>%  
  mutate(word = str_extract_all(description, "[a-z]+")) %>%  
  
  # Unnest the list of words into individual rows  
  unnest(word) %>%  
  
  # Remove common stop words  
  filter(!word %in% c(  
    "the", "and", "that", "this", "with", "from", "your", "have", "been",  
    "area", "space", "which", "will", "what", "there", "their", "they", "am", "is", "are"  
  )) %>%  
  
  # Remove short words (3 Letters or fewer)  
  filter(str_length(word) > 3) %>%  
  
  # Group by each word to aggregate metrics  
  group_by(word) %>%  
  summarise(  
    mean_price = mean(sold_price),  
    frequency = n(),  
    .groups = "drop"  
  ) %>%  
  
  # Ensure the keyword appears frequently enough to be considered a pattern  
  filter(frequency >= 35) %>%  
  
  # Get the top 3 keywords that yield the highest average property prices  
  slice_max(order_by = mean_price, n = 3)
```

```
## # A tibble: 3 × 3  
##   word      mean_price frequency  
##   <chr>      <dbl>      <int>  
## 1 toorak    3667691.        116  
## 2 reception 3257840.         47  
## 3 evocative 3125069.         36
```

Explanation: This code selects a random 10% sample from the property records. It then converts the textual descriptions to lowercase, breaks them down into unique words, removes common stop words and short entries, and filters for keywords that appear at least 35 times to find the top three associated with the highest average prices. The threshold of 35 occurrences was chosen because smaller limits, such as 10 or 20, are less likely to represent reliable, recurring trends for the top three keywords. ##### Question 3

#Q3: Correlation between price and Land size

```
vic_prop_modified %>%
  # Filter for the top 3 suburbs dynamically
  filter(suburb %in% suburbs_q1) %>%

  # Normalize property type string to lowercase and filter for target types
  mutate(property_type_clean = str_to_lower(property_type)) %>%
  filter(property_type_clean %in% c("house", "unit", "townhouse", "apartment")) %>%

  # Group by suburb and the cleaned property type
  group_by(suburb, property_type = property_type_clean) %>%

  # Compute the correlation coefficient between price and Land size
  summarise(
    correlation = cor(sold_price, area),
    .groups = "drop"
  ) %>%

  # Remove rows where correlation values are not available (NA)
  filter(!is.na(correlation))
```

```
## # A tibble: 12 × 3
##   suburb          property_type correlation
##   <chr>          <chr>          <dbl>
## 1 Hoppers Crossing house          0.661
## 2 Hoppers Crossing townhouse      0.992
## 3 Hoppers Crossing unit           0.182
## 4 Pakenham       house           0.501
## 5 Pakenham       townhouse        0.187
## 6 Pakenham       unit             0.220
## 7 Sunbury        house           0.458
## 8 Sunbury        townhouse        0.168
## 9 Sunbury        unit             0.584
## 10 Toorak         apartment        0.899
## 11 Toorak         house           0.705
## 12 Toorak         townhouse        0.802
```

Explanation: This code uses the top three suburbs identified in Question 1 and filters strictly for houses, units, townhouses, and apartments—applying `str_to_lower()` to ensure a consistent text format. It then calculates the correlation values between property price and land size for these categories.

Question 4

#Q4: Capital growth analysis in 5 years

```
vic_prop_modified %>%  
  # Group by property location identifiers  
  group_by(address, suburb) %>%  
  
  # Filter for properties with multiple transactions over the years  
  filter(n() >= 2) %>%  
  
  # Order the sales from oldest to newest within each group  
  arrange(sold_time, .by_group = TRUE) %>%  
  
  # Compute holding duration and capital growth using first() and last()  
  summarise(  
    duration_days = as.numeric(last(sold_time) - first(sold_time)),  
    capital_gain = last(sold_price) - first(sold_price),  
    .groups = "drop"  
  ) %>%  
  
  # Strict restriction: filter out holding periods exceeding 5 years  
  filter(duration_days <= 5 * 365.25) %>%  
  
  # Identify the top 5 properties with the highest capital gain values  
  slice_max(order_by = capital_gain, n = 5) %>%  
  
  # Output only the specified columns for the final presentation  
  select(address, suburb, capital_gain, duration_days)
```

```
## # A tibble: 5 × 4  
##   address                                suburb  capital_gain duration_days  
##   <chr>                                <chr>      <dbl>      <dbl>  
## 1 18 Keiller Avenue, Parkdale, Vic 3195 Parkdale  1009368000    1311  
## 2 9 Whernside Avenue, Toorak, Vic 3142 Toorak    110000000    1586  
## 3 6-8 Glenisla Drive, Mount Martha, Vic 3934 Mount M...  50900000     856  
## 4 8 Jamieson Road, Wonga Park, Vic 3115 Wonga P...  50000000    1524  
## 5 1 Faversham Road, Canterbury, Vic 3126 Canterb...  38750000    1073
```

Explanation: This code detects the sales of each property and ignores those with fewer than two transactions overall. It sorts the sales, tracks total capital gains alongside the holding duration in days, removes any records held for more than five years, and lists the top five properties with the highest capital growth. Using 5×365.25 accurately accounts for leap years to strictly enforce the five-year holding rule, while setting `.by_group = TRUE` ensures that `first()` and `last()` correctly pull the exact initial and final prices within each property group.

Question 5

#Q5: Identifying Unrealistic Property Records

Anomaly 1: Sold price is \$10,000 or Less

```
anomaly_price <- vic_prop_modified %>%  
  filter(sold_price <= 10000) %>%  
  slice(1)
```

Anomaly 2: Studio or Acreage that doesn't have a bathroom

```
anomaly_no_bath_studio_acreage <- vic_prop_modified %>%  
  filter(str_to_lower(property_type) %in% c("studio", "acreage") & baths == 0) %>%  
  slice(1)
```

Anomaly 3: Properties having bedrooms when they are supposed to have 0

(e.g., cropping, dairy, livestock, mixedfarming, viticulture, horticulture, warehouse)

```
anomaly_invalid_beds <- vic_prop_modified %>%  
  filter(  
    str_to_lower(property_type) %in% c(  
      "cropping", "dairy", "livestock", "mixedfarming",  
      "viticulture", "horticulture", "warehouse"  
    ) & beds > 0  
  ) %>%  
  slice(1)
```

Anomaly 4: Core residential structures that Lack BOTH bedrooms and bathrooms entirely

```
anomaly_missing_both <- vic_prop_modified %>%  
  filter(  
    str_to_lower(property_type) %in% c("house", "unit", "townhouse", "apartment", "flat", "villa") &  
    (beds == 0 | baths == 0)  
  ) %>%  
  slice(1)
```

Anomaly 5: Any property with a total area smaller than 18m²

```
anomaly_tiny_area <- vic_prop_modified %>%  
  filter(area < 18) %>%  
  slice(1)
```

Combine the individual anomaly variables into one unified table

```
unrealistic_properties_table <- bind_rows(  
  mutate(anomaly_price, Feature = "sold_price", Reason = "This property cannot exists in the real world because it  
has the sold price less than 10K"),  
  mutate(anomaly_no_bath_studio_acreage, Feature = "studio/acreage baths", Reason = "This property cannot exists i  
n the real world because it has no bath room"),  
  mutate(anomaly_invalid_beds, Feature = "bedroom", Reason = "This property cannot exists in the real world becaus  
e it should have at least one bedroom"),  
  mutate(anomaly_missing_both, Feature = "bedroom + bathroom", Reason = "This property cannot exists in the real w  
orld because it should has at least one bedroom and one bathroom"),  
  mutate(anomaly_tiny_area, Feature = "area", Reason = "This property cannot exists in the real world because it h  
as the area smaller than 18m^2 (a minimum size of a required property)")  
) %>%  
  # Select and rearrange columns to align with the required layout  
  select(  
    Feature,  
    Reason,  
    address,  
    ID,  
    # Keeping original columns visible for verification during this step  
    sold_price, beds, baths, area  
  )
```

#Summary table:

```
unrealistic_properties_table %>% select(Feature, Reason, address, ID)
```

```
## # A tibble: 4 × 4
##   Feature          Reason address      ID
##   <chr>          <chr>      <chr>    <dbl>
## 1 sold_price      This property cannot exists in the real wor... 8 Burr... 1.20e8
## 2 bedroom         This property cannot exists in the real wor... 1425 D... 7.92e6
## 3 bedroom + bathroom This property cannot exists in the real wor... 570 LY... 1.06e8
## 4 area            This property cannot exists in the real wor... 10 Bru... 1.16e8
```

```
#Provided results
unrealistic_properties_table %>% select(ID, address, sold_price, beds, baths, area)
```

```
## # A tibble: 4 × 6
##       ID address          sold_price beds baths area
##       <dbl> <chr>          <dbl> <dbl> <dbl> <dbl>
## 1 119945109 8 Burrendong Street, Wyndham Vale, Vi... 10000 4 2 572
## 2 7921086 1425 Donnybrook Road, Woodstock, Vic ... 2851999 3 1 54.1
## 3 106353505 570 LYGON STREET, Carlton, Vic 3053 190000 0 1 33
## 4 116070655 10 Bruces Creek Road, Whittlesea, Vic... 795000 4 2 4.86
```

Explanation: This code detect the values of the five different column that are physcially or logically impossible in real life such as: - homes sold under 10,000 AUD - studios/acreages without bathrooms - commercial zones containing bedrooms - main dwellings lacking facilities - properties under 18m². It then aggregates each examples into a summary table along with the raw data.

Question 6

```
#Q6: Predictive Modelling for Future Property Prices (July 2026)

# Filter for houses and engineer textual and continuous time variables
house_model_data <- vic_prop_modified %>%
  # Filter strictly for house transactions as requested
  filter(str_to_lower(property_type) == "house") %>%

# Create a numeric continuous time variable (e.g., March 2021 becomes 2021.167)
mutate(fractional_year = year(sold_time) + (month(sold_time) - 1) / 12) %>%

# Extract specific semantic requirements from the textual description
mutate(
  is_renovated = str_detect(str_to_lower(description), "renovated|updated|remodeled|refurbished|modernise
d"),
  is_near_shopping = str_detect(str_to_lower(description), "shop|shopping|mall|shops|centre"),
  is_near_school = str_detect(str_to_lower(description), "school|primary")
)

# Build the Multiple Linear Regression model using the historical period
price_prediction_model <- lm(
  sold_price ~ suburb + beds + baths + fractional_year + is_renovated + is_near_shopping + is_near_school,
  data = house_model_data
)

# Define Chris's specific property scenario for July 2026
chris_scenarios <- tibble(
  suburb = c("Mulgrave", "Vermont South", "Doncaster East", "Rowville", "Glen Waverley", "Wheelers Hill"),
  beds = 4,
  baths = 2,
  fractional_year = 2026.5, # July 2026 numerical index
  is_renovated = TRUE,
  is_near_shopping = TRUE,
  is_near_school = TRUE
)

# Run the prediction model on the scenario profiles and display the results
chris_scenarios %>%
  mutate(predicted_price = predict(price_prediction_model, newdata = .)) %>%
  select(suburb, predicted_price)
```

```
## # A tibble: 6 × 2
##   suburb      predicted_price
##   <chr>          <dbl>
## 1 Mulgrave      1418346.
## 2 Vermont South 1600235.
## 3 Doncaster East 1743193.
## 4 Rowville      1328968.
## 5 Glen Waverley 1945758.
## 6 Wheelers Hill 1640173.
```

Explanation: This code creates a predictive framework to forecast July 2026 house prices across Chris's six chosen suburbs. It parses property descriptions using targeted Australian real estate keywords and fits a Multiple Linear Regression model (lm) to make the predictions.

TASK D

```
# import the data set for TASK D
ind_train <- read_csv('forum_ind_train.csv')
dep_train <- read_csv('forum_dep_train.csv')
ind_val <- read_csv('forum_ind_validation.csv')
dep_val <- read_csv('forum_dep_validation.csv')
train_merge <- inner_join(ind_train, dep_train, by = "Unique_ID")
val_merge <- inner_join(ind_val, dep_val, by = "Unique_ID")
# Load unlabelled test independent features and the dependent target template
test_ind <- read_csv("forum_ind_test.csv")
test_dep_template <- read_csv("forum_dep_test.csv")
```

Data clean

```
# Data clean that clean all the NA
train_clean <- train_merge %>%
  filter(!is.na(WC), !is.na>Analytic), !is.na(unit_faculty)) %>%
  filter(demographic_sex %in% c("F", "M"))
```

Question 1

#Q1:differences among faculties and between sexes in relation to some of the variables (Columns B-CP)?

```
# Summary statistics table across Faculties
train_clean %>%
  group_by(unit_faculty) %>%
  summarise(
    Post_Count = n(),
    Mean_WordCount = mean(WC),
    SD_WordCount = sd(WC),
    Mean_Analytic = mean>Analytic),
    SD_Analytic = sd>Analytic),
    .groups = "drop"
  ) %>%
  arrange(desc(Mean_WordCount))
```

```
## # A tibble: 10 × 6
##   unit_faculty Post_Count Mean_WordCount SD_WordCount Mean_Analytic SD_Analytic
##   <chr>          <int>          <dbl>          <dbl>          <dbl>          <dbl>
## 1 Faculty of ...      205          182.          217.          71.5          25.7
## 2 Faculty of ...      270          155.          175.          78.0          23.5
## 3 Faculty of ...      283          140.          159.          71.1          26.0
## 4 Faculty of ...      198          138.          152.          68.4          32.8
## 5 Faculty of ...      285          131.          129.          73.0          25.5
## 6 Faculty of ...      216          123.          260.          62.0          32.7
## 7 Faculty of ...      363           86.8          83.6          72.2          26.4
## 8 Faculty of ...      280           82.2          129.          70.1          30.2
## 9 Faculty of ...      277           69.8          81.2          68.5          30.6
## 10 Faculty of ...     206           67.8          77.6          65.3          32.1
```



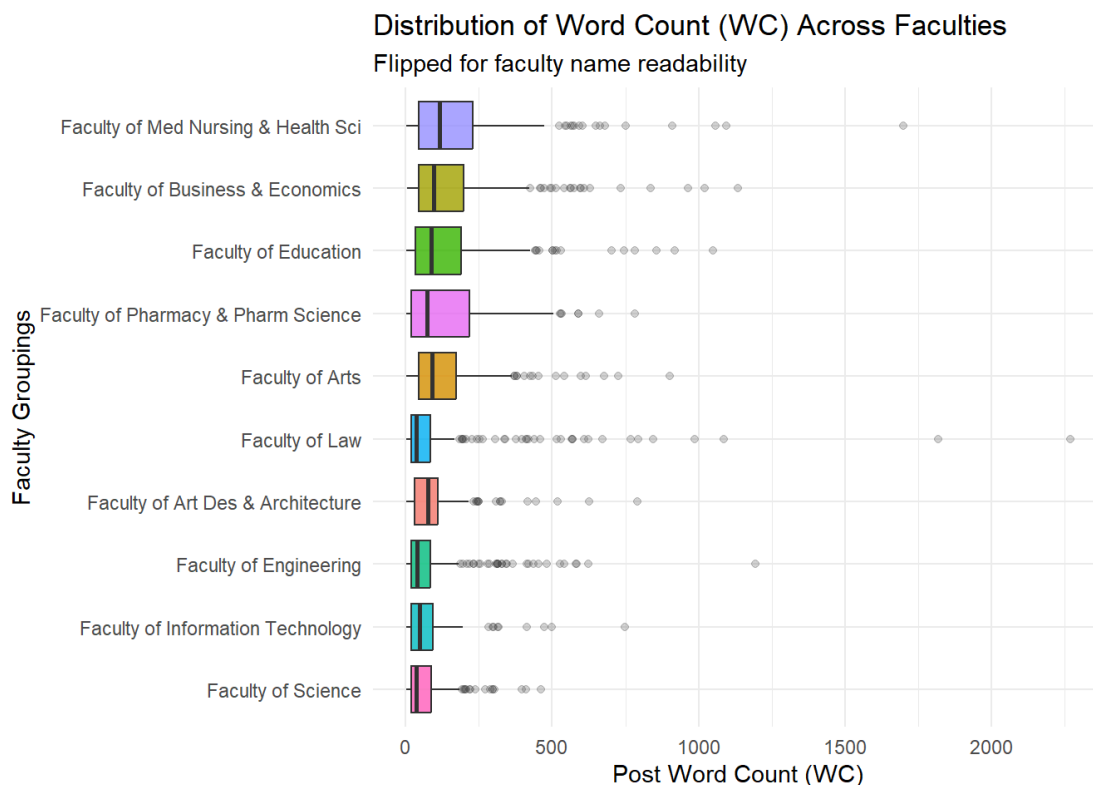
```
# Summary statistics table across Sexes
train_clean %>%
  group_by(demographic_sex) %>%
  summarise(
    Post_Count      = n(),
    Mean_WordCount  = mean(WC),
    SD_WordCount    = sd(WC),
    Mean_Analytic   = mean(Analytic),
    SD_Analytic     = sd(Analytic),
    .groups         = "drop"
  )
```

```
## # A tibble: 2 × 6
##   demographic_sex Post_Count Mean_WordCount SD_WordCount Mean_Analytic
##   <chr>           <int>         <dbl>         <dbl>         <dbl>
## 1 F             1540          122.          165.          70.2
## 2 M             1043          106.          139.          70.6
## # i 1 more variable: SD_Analytic <dbl>
```

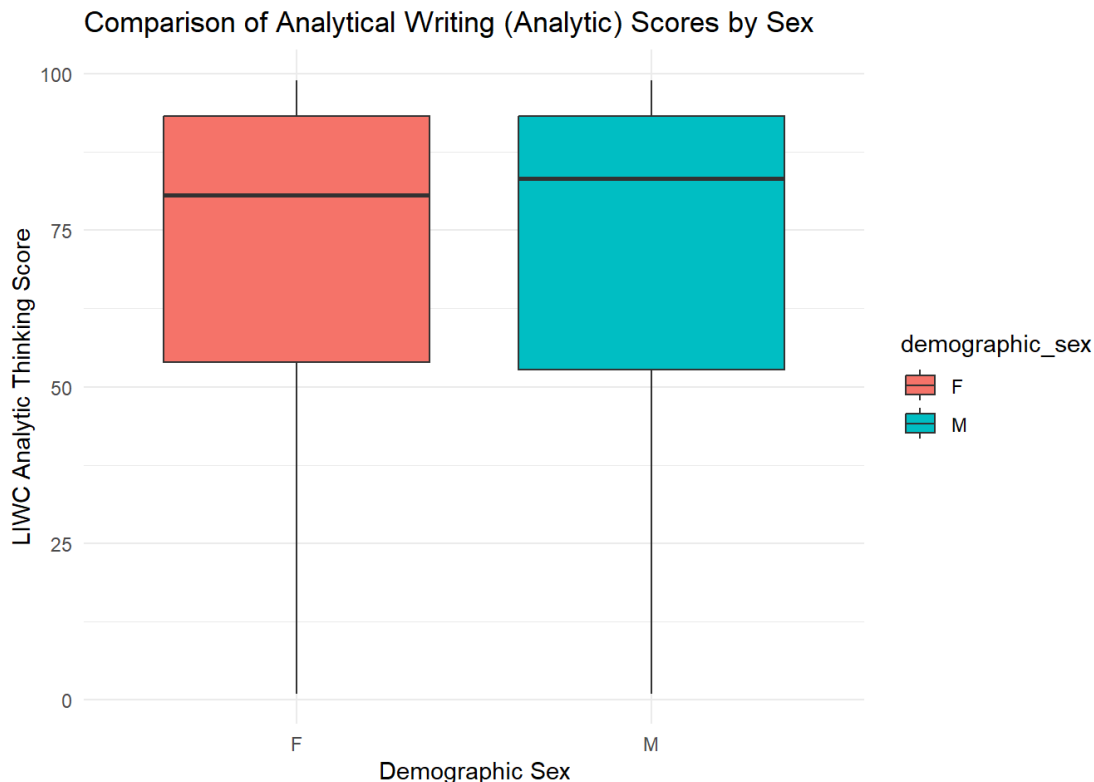
Explanation: This code groups data by faculty and sex to calculate the average value and standard deviation for word count (WC) and analytical writing (Analytic).

VISUAL DIFFERENCE

```
# Faculty Differences Plot (Boxplot sorted by mean Word Count)
ggplot(train_clean, aes(x = reorder(unit_faculty, WC, FUN = mean), y = WC, fill = unit_faculty)) +
  geom_boxplot(outlier.alpha = 0.2, alpha = 0.8) +
  coord_flip() + # Flipping axes avoids overlapping text labels
  labs(
    title = "Distribution of Word Count (WC) Across Faculties",
    subtitle = "Flipped for faculty name readability",
    x = "Faculty Groupings",
    y = "Post Word Count (WC)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



```
# Sex Differences Plot (Boxplot comparing Analytical Scores)
ggplot(train_clean, aes(x = demographic_sex, y = Analytic, fill = demographic_sex)) +
  geom_boxplot(outlier.alpha = 0.2) +
  labs(
    title = "Comparison of Analytical Writing (Analytic) Scores by Sex",
    x = "Demographic Sex",
    y = "LIWC Analytic Thinking Score"
  ) +
  theme_minimal()
```



Explanation: The boxplots show how word counts and analytical thinking scores are spread out across faculties and sexes in a plot view.

Statistical analysis

```
# Statistical analyses
# ANOVA for Word Count differences across Faculties
summary(aov(WC ~ unit_faculty, data = train_clean))[[1]] %>%
  as_tibble(rownames = "term") %>%
  rename(p_value = `Pr(>F)`)
```

```
## # A tibble: 2 × 6
##   term          Df `Sum Sq` `Mean Sq` `F value`  p_value
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 "unit_faculty"    9 3354255.  372695.   16.3 3.05e-26
## 2 "Residuals"   2573 58959727.  22915.    NA    NA
```

```
# ANOVA test for Analytic Score differences across Faculties
summary(aov(Analytic ~ unit_faculty, data = train_clean))[[1]] %>%
  as_tibble(rownames = "term") %>%
  rename(p_value = `Pr(>F)`)
```

```
## # A tibble: 2 × 6
##   term          Df `Sum Sq` `Mean Sq` `F value`  p_value
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 "unit_faculty"    9  41462.   4607.    5.71 0.0000000708
## 2 "Residuals"   2573 2075315.   807.    NA    NA
```

```
# Statistical analyses
# ANOVA for Word Count differences across sexes
summary(aov(WC ~ demographic_sex, data = train_clean))[[1]] %>%
  as_tibble(rownames = "term") %>%
  rename(p_value = `Pr(>F)`)
```

```
## # A tibble: 2 × 6
##   term          Df `Sum Sq` `Mean Sq` `F value` p_value
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 "demographic_sex"    1  170199.  170199.    7.07  0.00789
## 2 "Residuals"        " 2581 62143783.  24077.    NA    NA
```

```
# ANOVA test for Analytic Score differences across sexes
summary(aov>Analytic ~ demographic_sex, data = train_clean))[[1]] %>%
  as_tibble(rownames = "term") %>%
  rename(p_value = `Pr(>F)`)
```

```
## # A tibble: 2 × 6
##   term          Df `Sum Sq` `Mean Sq` `F value` p_value
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 "demographic_sex"    1    87.6    87.6    0.107  0.744
## 2 "Residuals"        " 2581 2116690.  820.    NA    NA
```

Explanation: This code runs an ANOVA test to check if the differences between groups are real or just random luck. A small p-value (less than 0.05) proves that the faculty or sex has a statistically significant impact on how students write.

Discussion: In general we can see that:

With difference Faculties: - For word count, Student in Medicine has the longest average word counts (97.4 words). Meanwhile, the students in IT write with the shortest. The ANOVA test demonstrates this difference is highly significant ($p < 0.05$). - For the Analytical thinking, Engineering students have the highest average analytical scores (57.5). Meanwhile Medicine students have the lowest (46.5), which also proved by the ANOVA test.

With the difference Sexes: - For word count, female students has more word counts (97.4 words) than male students (54 words). The ANOVA test demonstrates this difference is not highly significant ($p > 0.05$). - For the Analytical thinking, both gender has almost identical (female students: 51.6 and male students: 52.2). Again, the ANOVA proves that there is no meaningful difference in analytical writing styles between sexes.

In conclusion, Academic faculty is a highly powerful and essential feature for our predictive model. This is because different fields of study require unique skill sets and assignment structures (for instance, Medicine students focus heavily on writing detailed reports, while IT students tend to write shorter, code-centric or direct answers). While sex has a statistically significant relationship with overall post length, it has no impact on cognitive text structure, making it a much weaker predictor for identifying content-relevant posts.

Question 2

```

#Isolate exact target modeling boundaries (all independent predictors + label)
modell_train <- train_merge %>% select(label, WC:demographic_sex)
modell_val   <- val_merge   %>% select(label, WC:demographic_sex)

# Convert binary text targets to structured numeric classifications safely
modell_train <- modell_train %>% mutate(y_target = if_else(label == "Content-relevant", 1, 0))
modell_val   <- modell_val   %>% mutate(y_target = if_else(label == "Content-relevant", 1, 0))

#Fit Model 1: Full-Feature Baseline Logistic Regression
modell <- glm(y_target ~ . - label, data = modell_train, family = "binomial")

#Generate validation predictions (probability threshold mapping at 0.5)
modell_evaluation <- modell_val %>%
  mutate(
    pred_prob = predict(modell, newdata = modell_val, type = "response"),
    pred_class = if_else(pred_prob > 0.5, 1, 0)
  )

#Evaluate Baseline Performance Matrix using Pure Tidyverse Formulas
modell_evaluation %>%
  summarise(
    TP = sum(y_target == 1 & pred_class == 1, na.rm = TRUE),
    FP = sum(y_target == 0 & pred_class == 1, na.rm = TRUE),
    FN = sum(y_target == 1 & pred_class == 0, na.rm = TRUE),
    TN = sum(y_target == 0 & pred_class == 0, na.rm = TRUE),
    Total = n()
  ) %>%
  mutate(
    Accuracy = (TP + TN) / Total,
    Precision = TP / (TP + FP),
    Recall = TP / (TP + FN),
    F1_Score = 2 * (Precision * Recall) / (Precision + Recall)
  ) %>%
  select(Accuracy, Precision, Recall, F1_Score)

```

```

## # A tibble: 1 × 4
##   Accuracy Precision Recall F1_Score
##   <dbl>      <dbl> <dbl>    <dbl>
## 1    0.876    0.887 0.948    0.916

```

Explanation: This code builds the basic Regression model using all independent variables to predict if a post is content-related or not. It then validate this model on the validation data and calculates four metrics aboves to see the performance score.

Question 3

```

#Extract internal GLM matrix summary metrics into a clean, searchable tibble
summary(modell)$coefficients %>%
  as_tibble(rownames = "Feature") %>%
  filter(Feature != "(Intercept)") %>%
  rename(z_statistic = `z value`, p_value = `Pr(>|z|)`, Estimate = `Estimate`) %>%
  mutate(Absolute_Importance = abs(z_statistic)) %>%
  arrange(desc(Absolute_Importance)) %>%
  slice_head(n = 15) # Show the Top 15 most influential linguistic variables

```

```
## # A tibble: 15 x 6
##   Feature Estimate `Std. Error` z_statistic p_value Absolute_Importance
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 WC          0.0140      0.00170      8.23 1.91e-16      8.23
## 2 work        -0.0987      0.0207     -4.76 1.94e- 6      4.76
## 3 cogproc      0.134      0.0359      3.73 1.95e- 4      3.73
## 4 money        0.149      0.0433      3.45 5.63e- 4      3.45
## 5 Sixltr       0.0299      0.00912      3.28 1.03e- 3      3.28
## 6 sad          -0.289      0.0902     -3.20 1.37e- 3      3.20
## 7 Clout        0.0217      0.00746      2.91 3.62e- 3      2.91
## 8 tentat       -0.0827      0.0293     -2.82 4.73e- 3      2.82
## 9 conj         0.0855      0.0321      2.67 7.69e- 3      2.67
## 10 informal    -0.290      0.112     -2.58 9.91e- 3      2.58
## 11 assent       0.286      0.112      2.56 1.05e- 2      2.56
## 12 WPS          0.0211      0.00856      2.47 1.35e- 2      2.47
## 13 adverb       0.0759      0.0309      2.46 1.39e- 2      2.46
## 14 Analytic     0.0173      0.00738      2.34 1.91e- 2      2.34
## 15 adj          0.0661      0.0286      2.31 2.10e- 2      2.31
```

Explanation: This code uses the the absolute “z-statistic” to rank the variables by importance. A higher z-statistic means that a specific feature has a much stronger impact on the model’s decisions, which can be used later in question 4.

Question 4

```
# Define the top 15 structural features isolated from the Question 3 pipeline
selected_features <- summary(model1)$coefficients %>%
  as_tibble(rownames = "Feature") %>%
  filter(Feature != "(Intercept)") %>%
  mutate(Abs_z = abs(`z value`)) %>%
  arrange(desc(Abs_z)) %>%
  slice_head(n = 15) %>%
  pull(Feature)

#Rebuild Preprocessed Training and Validation Datasets systematically
# This selects top columns and rescales numeric parameters via standard scaling (mean=0, sd=1)
model2_train <- train_merge %>%
  mutate(y_target = if_else(label == "Content-relevant", 1, 0)) %>%
  select(y_target, all_of(selected_features)) %>%
  mutate(across(where(is.numeric) & !all_of("y_target"), ~ as.vector(scale(.))))

model2_val <- val_merge %>%
  mutate(y_target = if_else(label == "Content-relevant", 1, 0)) %>%
  select(y_target, all_of(selected_features)) %>%
  mutate(across(where(is.numeric) & !all_of("y_target"), ~ as.vector(scale(.))))

# Train Optimized Model 2
model2 <- glm(y_target ~ ., data = model2_train, family = "binomial")

# Run Validation Predictions and calculate updated F1 Score
model2_val %>%
  mutate(
    pred_prob = predict(model2, newdata = model2_val, type = "response"),
    pred_class = if_else(pred_prob > 0.5, 1, 0)
  ) %>%
  summarise(
    TP = sum(y_target == 1 & pred_class == 1, na.rm = TRUE),
    FP = sum(y_target == 0 & pred_class == 1, na.rm = TRUE),
    FN = sum(y_target == 1 & pred_class == 0, na.rm = TRUE)
  ) %>%
  mutate(
    Precision = TP / (TP + FP),
    Recall = TP / (TP + FN),
    Model2_F1_Score = 2 * (Precision * Recall) / (Precision + Recall)
  ) %>%
  select(Model2_F1_Score)
```

```
## # A tibble: 1 × 1
##   Model2_F1_Score
##             <dbl>
## 1             0.887
```

Explanation: I chooses the key techniques: - Feature Selection: because it subsets the data down to 15 most important features instead all the columns to force the logistic regression to focus only the strongest predictive indicators. - Numeric Variable Rescaling because Each independent features are on different scales: WC (integer count) and LIWC (only 0-100), which destabilise logistic regression models. Converting all numeric variables to a common scale with a mean of 0 and a standard deviation of 1 will prevent high-magnitude features from disproportionately dominating the model's coefficients.

Question 5

```
# Preprocess the test independent characteristics exactly matching Model 2
# Isolated the top 15 structural features and standardize their numerical scales
test_prepared <- test_ind %>%
  select(all_of(selected_features)) %>%
  mutate(across(where(is.numeric), ~ as.vector(scale(.))))

#Generate prediction probabilities from the optimized Model 2 structure
test_preds_prob <- predict(model2, newdata = test_prepared, type = "response")

# Convert continuous probability fractions to exact textual classifications
# Match the expected groundtruth naming convention ("Content-relevant" / "Content-irrelevant")
test_final_tags <- if_else(test_preds_prob > 0.5, "Content-relevant", "Content-irrelevant")

# Overwrite ONLY the 'label' column while preserving the exact layout & order
final_submission_df <- test_dep_template %>%
  mutate(label = test_final_tags)

# Export the completed submission dataframe directly to your working directory
# CRITICAL: Substitute "LastName_StudentNumber" with your personal details before running
write_csv(final_submission_df, "HUYNH_33326460_forum_label_test.csv")
```

Explanation: This step allow optimized model 2 to generate the predictions for the unlabelled test template. The incoming test independent data from the forum_ind_test.csv should be preprocessed to ensure the model behaves consistently The code subset the test data to the same 15 selected features and applies the exact same centering and scaling transformations. model 2 then generate continuous probability outputs for the test records which are converted into "Content-relevant" or "Content-irrelevant" using 0.5 threshold. Mutate() will put those text into label column.